

The Stone House

Арбен получил в наследство прекрасный участок земли на холмах, возвышающихся над Гирокастрой. Вдохновлённый знаменитыми городскими домами с каменными крышами, он собирается построить свой собственный традиционный дом.

Участок представлен в виде квадратной таблицы $N \times N$, строки и столбцы которой пронумерованы от 1 до N . У каменистой местности есть одна особенность: в каждой строке и в каждом столбце таблицы находится ровно один большой несдвигаемый валун.

Чтобы заложить фундамент своего дома, Арбен должен выбрать **пустую квадратную подтаблицу** — область размера $S \times S$, не содержащую ни одного валуна.

Пусть M — наибольшее целое число такое, что при *любом* расположении валунов, удовлетворяющем приведённым выше условиям, гарантированно существует пустая квадратная подтаблица размера $M \times M$. Арбен хочет найти *какую-нибудь* пустую квадратную подтаблицу размера не менее $M \times M$.

Чтобы обследовать участок, Арбен просит помощи у своей подруги Бесы. Беса принесла специальное геодезическое устройство под названием **Gurëmatës** ("счётчик камней").

За одну операцию Беса может настроить Gurëmatës на сканирование прямоугольной подтаблицы, заданной своим левым верхним углом (x_1, y_1) и правым нижним углом (x_2, y_2) . Устройство возвращает точное количество валунов в этой области. Gurëmatës питается от маленькой батарейки, и каждое сканирование расходует много энергии, поэтому Беса должна использовать его как можно меньшее число раз.

Напишите программу, которая с помощью Gurëmatës находит пустую квадратную подтаблицу размера $S \times S$ при $S \geq M$ и сообщает её расположение.

Детали реализации

Вы должны реализовать следующую процедуру:

```
std::vector < int > solve(int N);
```

- N : размер таблицы.

- Процедура должна вернуть ровно четыре целых числа $\{x_1, y_1, x_2, y_2\}$: левый верхний и правый нижний углы найденного пустого квадратного подпрямоугольника. Должно выполняться условие $x_2 - x_1 = y_2 - y_1 = S - 1$ при $S \geq M$.
- Эта процедура вызывается ровно один раз для каждого теста.

Эта процедура может вызывать следующую процедуру:

```
int ask(int x1, int y1, int x2, int y2);
```

- x_1, y_1 : строка и столбец левого верхнего угла сканируемого прямоугольника.
- x_2, y_2 : строка и столбец правого нижнего угла сканируемого прямоугольника.
- Должны выполняться условия $1 \leq x_1 \leq x_2 \leq N$ и $1 \leq y_1 \leq y_2 \leq N$. Иначе выполнение вашей программы немедленно прекращается, и она получает 0 баллов за этот тест.
- Процедура возвращает количество валунов в отсканированном прямоугольнике.

Проверяющая программа не является адаптивной: позиции валунов фиксируются до вызова `solve` и не зависят от ваших вызовов `ask`.

Ограничения

- $2 \leq N \leq 500$
- Вы можете сделать не более $2.5 \cdot 10^5$ вызовов функции `ask`.

Подзадачи

Подзадача	Баллы	Дополнительные ограничения
1	15	$N \leq 40$
2	25	$N \leq 150$
3	60	Без дополнительных ограничений.

Оценивание

Если `solve` возвращает недопустимую область, делает недопустимый вызов `ask` или вызывает `ask` более $2.5 \cdot 10^5$ раз, вы получаете 0 баллов за этот тест.

Иначе пусть Q — число вызовов `ask`. За этот тест вы получаете баллы подзадачи, умноженные на $\min(1, f(Q))$, где

$$f(Q) = \begin{cases} \max(0, -0.05 \cdot Q + 1.55), & \text{if } Q \leq 15 \\ \max(0, -0.19 \cdot \log_{10}(Q) + 1.02), & \text{if } Q > 15 \end{cases}$$

Ваш балл за подзадачу — это минимальный балл по всем тестам этой подзадачи. Итоговый балл — сумма баллов за подзадачи.

Чтобы помочь вам оценить свой результат, в таблице ниже приведён приблизительный множитель для некоторых конкретных значений Q :

Количество запросов (Q)	Множитель ($\approx$)
$Q \leq 11$	1.00
12	0.95
15	0.80
50	0.70
100	0.64
1000	0.45
10000	0.26
100000	0.07
$Q \geq 233573$	0.00

Тестовый грайдер

Тестовый грайдер читает входные данные в следующем формате:

```
N
c_1 c_2 ... c_N
```

Здесь c_i — столбец валуна в строке i ($1 \leq c_i \leq N$).

Тестовый грайдер выводит одну строку, содержащую множитель $\min(1, f(Q))$, полученный за ваши запросы, или 0, если возвращённая подсетка или какой-либо вызов `ask` недопустим.

Примеры

Пример 1

Входные данные тестового грейдера:

```
4
1 2 3 4
```

Валуны находятся в клетках (1, 1), (2, 2), (3, 3) и (4, 4). Ваша программа этого не знает.

Грейдер вызывает `solve(4)`. Одна из возможных последовательностей вызовов:

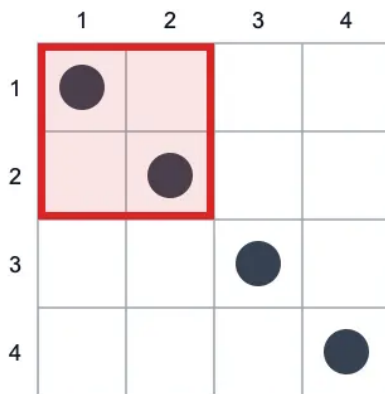
- `ask(1, 1, 2, 2)` возвращает 2.
- `ask(2, 4, 4, 4)` возвращает 1.
- `ask(3, 1, 3, 2)` возвращает 0.

Затем `solve` возвращает `{1, 3, 2, 4}`.

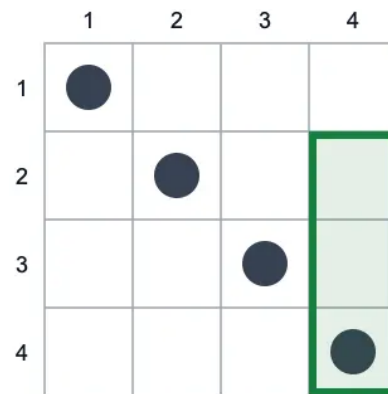
Выходные данные тестового грейдера:

```
1
```

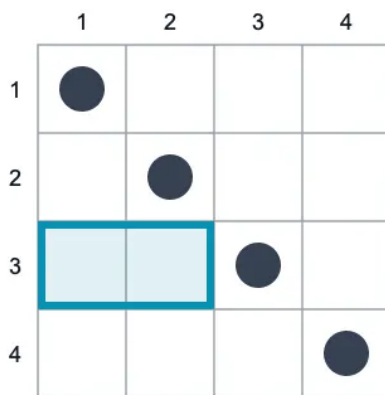
Первый вызов сканирует левый верхний блок 2×2 и находит валуны в клетках (1, 1) и (2, 2). Второй сканирует строки с 2-й по 4-ю в столбце 4 и находит валун в клетке (4, 4). Третий сканирует первые две клетки строки 3 и не находит валунов. Возвращённая область покрывает строки с 1-й по 2-ю и столбцы с 3-го по 4-й, что является пустой подсеткой 2×2 .



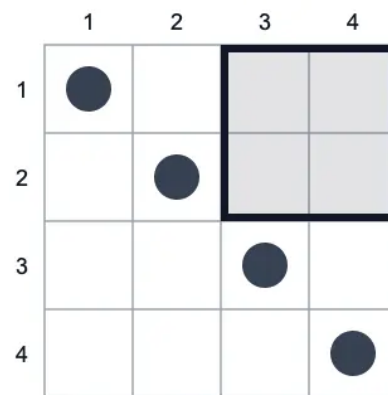
ask(1, 1, 2, 2) returns 2



ask(2, 4, 4, 4) returns 1



ask(3, 1, 3, 2) returns 0



return {1, 3, 2, 4}

При $N = 4$ программа использовала $Q = 3$ вызова, поэтому $f(Q) = 1.4$, и множитель равен 1. Другие ответы также принимаются, например $\{3, 1, 4, 2\}$, $\{1, 2, 1, 2\}$, $\{3, 2, 3, 2\}$, $\{3, 4, 3, 4\}$ или любая пустая клетка, так как $M = 1$ при $N = 4$.